

Bedtime Story Generator — Module 7 Deploy Guide

Swarup Biswas

2026-07-15

Contents

Deploy Guide — Bedtime Story Generator	1
Phase 0 — Push your bedtime app to YOUR GitHub (~5 minutes)	1
Phase 1 — Render (backend + managed Postgres)	3
Phase 2 — Vercel (static frontend)	6
Phase 3 — Wire them together	7
Phase 4 — Verify on a phone	8
Common gotchas	8
What's NOT in V1's deploy	10
After you finish	10
Doc verification provenance	10

Deploy Guide — Bedtime Story Generator

This is the click-by-click companion to `docs/crash_course.md` Module 7. The crash course explains why the deploy is shaped the way it is (Vercel for the static frontend, Render for the long-running backend + managed Postgres, no Vercel-everything-serverless). This guide walks the how.

Before you start. Make sure all three are ready: (1) free-tier accounts on both `render.com` and `vercel.com`, each signed up with your GitHub account (Render and Vercel need GitHub access to read your repo); (2) your `GEMINI_API_KEY` from `https://aistudio.google.com/apikey` ready to paste — you'll paste it once, into Render's web UI, and never into a file or a commit; (3) the bedtime app code you've been working in. Phase 0 below covers pushing that code to your GitHub so Render and Vercel can read it.

The whole deploy is ~15 minutes if everything cooperates, ~25 if Render's build queue is slow. There are 5 phases: get your code on your GitHub, provision Render, provision Vercel, wire them together, verify on a phone.

Phase 0 — Push your bedtime app to YOUR GitHub (~5 minutes)

Render and Vercel deploy by reading from a GitHub repo they have access to — typically a repo in your own GitHub account. Your local cohort clone (`bedtime-story-generator-cohort`) lives on the instructor's account; you can't push to it directly. So before configuring Render or Vercel, you push the code to your GitHub.

You have two clean paths — pick whichever feels natural:

Path A — Fork the cohort repo (simplest)

One command via the GitHub CLI. From inside your cohort clone:

```
# From inside the cohort repo root (~/.code/bedtime-story-generator-cohort or similar):
gh auth login          # only the first time per machine; pick GitHub.com → HTTPS →
↪ login with browser
gh repo fork --remote-name=origin --clone=false
git push -u origin main
```

That's it. `gh repo fork` creates `<your-username>/bedtime-story-generator-cohort` on your account and switches the local clone's origin remote to point there. The `git push` puts the current state of your code on your fork.

WSL2 students: `gh auth login` will try to open a browser. On WSL2 the browser-open sometimes doesn't bridge to Windows automatically — if it doesn't, the terminal prints a URL like `https://github.com/login/device` and an 8-character code. Copy the URL into your Windows browser manually, paste the code, approve. The device-flow path works either way.

Verify: `gh repo view --web` opens your fork in the browser. The page should show the cohort code under your GitHub username.

Path B — Fresh repo (clean history, slightly more setup)

Use this if you'd rather have an unambiguously-your repo with no inherited cohort commits:

```
# From inside the cohort repo root:
rm -rf .git
git init && git branch -M main
git add . && git commit -m "Initial commit --- my bedtime story app"
gh auth login          # only the first time per machine
gh repo create <your-repo-name> --public --source=. --remote=origin --push \
  --description "Bedtime Story Generator --- SCTP course project"
```

Pick `<your-repo-name>` carefully — Render and Vercel will use it in their dashboards. `bedtime-story-app-<your-name>` or similar is fine.

Verify: `gh repo view --web`. Same check as Path A — the page should show your code on your account.

Common gotchas at Phase 0

Symptom	Cause	Fix
<code>gh</code> : command not found	GitHub CLI not installed	macOS: <code>brew install gh</code> . WSL2 Ubuntu: install via the keyring-file apt flow (see the FastAPI course's <code>docs/setup_walkthrough.md</code>).
<code>gh auth login</code> browser doesn't open	WSL2 browser bridge inconsistent	Use the device-flow URL the terminal prints; paste into your Windows browser manually.
<code>git push</code> rejected with Permission denied (publickey)	<code>gh auth login</code> picked SSH but the remote is HTTPS (or vice versa)	Re-run <code>gh auth login</code> and match protocol to remote. Or <code>git remote set-url origin https://github.com/<user>/<repo>.git</code> to force HTTPS.

Symptom	Cause	Fix
gh repo fork fails with “fork already exists”	You forked once already; the fork is on your account	Skip the fork step. <code>git remote set-url origin https://github.com/<your-user>/bedtime-story-generator-cohort.git</code> then <code>git push -u origin main</code> .
Render/Vercel can't see your new repo in their import lists	You signed up for Render/Vercel with the wrong GitHub account, OR the repo is private and you didn't grant access	In Render/Vercel: settings → GitHub integration → grant access to the specific repo (or “all repositories”).

Don't move to Phase 1 until **gh repo view --web** shows your code on your GitHub account. Render and Vercel will ask you to pick a repo from a list — that list is whatever YOU own on GitHub, not what's in your local terminal.

Phase 1 — Render (backend + managed Postgres)

Render reads `render.yaml` at the repo root and provisions both the web service (FastAPI) and a managed Postgres in one go. You don't click “create database” separately.

1.1 Create the Blueprint

1. Sign in to <https://dashboard.render.com>.
2. Top-right: New → Blueprint.
3. Connect your GitHub account if Render hasn't asked yet. Grant access to the repo (`bedtime-story-generator` or whatever you named it).
4. Pick the repo from the list. Render scans the repo and finds `render.yaml`.
5. Blueprint name: type something self-descriptive. For your first test deploy, **bedtime-story-test** is a good name — it labels this Blueprint as your throwaway test in your dashboard, separate from the real cohort version you'll create later. Render uses this name only in your own dashboard listing; it does NOT become part of any URL.
6. Click Apply. Render now starts provisioning.

1.2 Watch the build

Render shows two cards appear in the Blueprint:

- **bedtime-story-api** (Web Service) — the FastAPI backend. Build takes ~3–5 minutes the first time because Render runs `pip install -r requirements.txt` against the course-wide union (`fastapi`, `uvicorn`, `jinja2`, `httpx`, `psycpg`, `python-dotenv`, `google-genai` — all fetched from PyPI in Render's build container).
- **bedtime-story-db** (Postgres database) — usually online in ~30 seconds (empty Postgres provisions fast).

Don't click anything yet. Open each card to watch the build log if you want to narrate to yourself what's happening.

Free-tier caveats (verified against Render docs in 2026): - Free Postgres expires 30 days after creation. After 30 days the database is inaccessible until you upgrade to a paid plan; there's a 14-day grace period to upgrade before deletion. So a free Postgres provisioned today is reachable until day 30, in a “must-upgrade” state from day 30 to day 44, and gone entirely from day 45. - Free web services are capped at 750 instance-hours per workspace per calendar month. A single always-running free service uses ~720 hours/month, leaving ~30 hours of headroom. Multiple free web services in one workspace will trip this cap. Render suspends all free services until the next month when the cap

is hit. - Free web services spin down after 15 minutes of no traffic. First request after spin-down takes ~30 seconds (cold start). This is correct behaviour, not a bug.

For a one-week test deploy or a 4-week cohort, all three are fine. For longer-running classroom use or anything resembling production, upgrade to a paid Postgres tier.

1.3 Set **GEMINI_API_KEY**

`render.yaml` declares the key with `sync: false`, which tells Render “a human supplies this value; never sync it from source control.” That’s the production-correct pattern: real keys never enter git.

There are two moments at which Render asks you for the value, and either works:

Moment A — During Blueprint creation (most common path). When you click Apply on the Blueprint, Render scans `render.yaml`, sees the `sync: false` variable, and prompts you for the value before provisioning starts. Paste your real Google AI Studio key into that prompt. Render won’t proceed until you give a value (or explicitly skip — see below). The key is stored in Render’s encrypted env store and injected into the service at runtime.

Moment B — After provisioning, in the Environment tab. If you skipped the prompt during Blueprint creation (or used a placeholder), the service will fail to start: `app.services.gemini_service` reads the key at import time, so `uvicorn` raises `KeyError: 'GEMINI_API_KEY'` before the port binds and the deploy never reaches Live (V1’s loud-fail discipline, now in production). In Render this shows as a failed deploy / crash-looping service in the Logs, not a running service that errors on a request. Fix:

1. Click into the **bedtime-story-api** service card.
2. Left sidebar → Environment.
3. Find the row with key `GEMINI_API_KEY` and an empty/placeholder value.
4. Click Edit, paste your real key, Save Changes.
5. Render auto-redeploys (~30 seconds — much faster than the first build because the venv is cached).

Important: the per-Blueprint-creation prompt only fires once, at initial creation. Future re-applications of the same Blueprint silently skip `sync: false` variables — the value you set is preserved. So Moment A is a one-shot opportunity; if you miss it, Moment B is the canonical place to fix it from then on.

1.4 Apply the schema migration

The `stories` table doesn’t exist yet — Render provisioned an empty Postgres. You need to run Module 6’s migration once. The Render-documented path is to copy the database’s external URL from the dashboard, then run `psql` from your local terminal against that URL.

1. From your Blueprint, click into the **bedtime-story-db** card.
2. Top-right of the database page, click Connect. Render shows two URLs:
 - Internal Database URL — only reachable from inside Render’s network. Used by your web service automatically (this is what `fromDatabase` wires).
 - External Database URL — reachable from anywhere on the internet, including your laptop. Use this one for the migration. Click the copy icon next to it.
3. From your local terminal, run the migration. The migration file lives in two places (they’re byte-identical):
 - `sql/002_create_stories.sql` — relative path from inside **dist/module_06_story_library/** or **dist/module_07_deploy_vercel/** (each module’s `dist` folder is self-contained and carries its own `sql/` directory).
 - `dist/module_07_deploy_vercel/sql/002_create_stories.sql` — relative path from the repo root.

Use whichever matches where you’re currently `cd`ed. Quote the URL — it contains special characters:

```
# If your terminal is in dist/module_06_story_library/ or
↪ dist/module_07_deploy_vercel/:
psql "<paste-the-External-Database-URL-here>" -f sql/002_create_stories.sql

# If your terminal is at the repo root:
psql "<paste-the-External-Database-URL-here>" -f
↪ dist/module_07_deploy_vercel/sql/002_create_stories.sql
```

(macOS / Linux / WSL2 Ubuntu — same commands; forward slashes everywhere.)

If you see `psql: error: ... No such file or directory`, your current working directory doesn't have the path you specified. Either `cd` to the right folder or use the path that matches where you are.

You should see three lines of output: `DROP TABLE`, `CREATE TABLE`, `CREATE INDEX`. If `DROP TABLE` reports a "table does not exist" notice followed by `DROP TABLE`, that's fine — `IF EXISTS` makes that case a no-op.

4. Confirm the table is empty:

```
psql "<External-Database-URL>" -c "SELECT * FROM stories;"
# Expected: (0 rows)

psql "<External-Database-URL>" -c "\d stories"
# Expected: shows the eight columns + idx_stories_child_name_created_at index
```

Why **psql** from local terminal, not a web shell? Render's public docs document the external `psql` command flow as the canonical way to run SQL against a managed Postgres. If your dashboard also shows an in-browser shell, that works too — but the external-`psql` path is the one we can rely on across UI changes.

Security warning — the External Database URL contains your database password

The string Render gave you in the Connect menu looks like:

```
postgresql://USERNAME:PASSWORD@HOST/DATABASE
```

The `PASSWORD` chunk is a real, valid credential. Anyone who sees that URL can connect to your Postgres and read/write/drop your data.

Treat the External URL like an API key: - Copy it from Render's UI directly into your terminal — don't pass through any other surface (no chat apps, no shared documents, no Slack messages, no screenshots, no commit messages, no echo into a file you might git-add later). - Don't paste it back as a verification step ("here's my command, did I get it right?"). Paste only the command shape with `<URL>` placeholder; never the resolved version. - Render rotates this URL only on explicit request — a leaked URL stays valid until you rotate.

If you accidentally leak the URL (paste it into chat, commit it, screenshot it, etc.):

1. Open the **bedtime-story-db** card in Render → top-right Connect menu (or the `:` more-actions menu).
2. Find Rotate Credentials (label varies — also seen as Rotate Password). Confirm rotation.
3. Render generates a new password; the old one becomes invalid immediately. Anyone holding the leaked URL can no longer connect.
4. The web service `bedtime-story-api` auto-redeploys with the new internal credential — `from-Database` re-injects automatically; you don't update anything in the service env.
5. Copy the new External URL from the Connect menu for any further `psql` commands.

Rotation takes ~30 seconds and costs nothing. When in doubt, rotate.

1.5 Smoke-test the backend (Phase 1 gate)

This is your gate before Phase 2. Don't move to Vercel until this passes — if `/healthz` doesn't return cleanly, the frontend you're about to deploy will hit a broken backend, and the bug will look like a Vercel/CORS problem when it's actually Render.

Once `bedtime-story-api` shows status `Live` in the dashboard, copy its URL from the top of the service card (something like `https://bedtime-story-api-XXXX.onrender.com` — Render appends a unique suffix per service). Then in your local terminal:

```
curl -s https://<your-render-url>/healthz
# Expected: {"postgres":true}
```

Three possible outcomes:

Output	Meaning	Next step
<code>{"postgres":true}</code>	Backend live, database wired, migration applied.	✓ Phase 1 complete. Save the Render URL — you'll paste it into <code>frontend/script.js</code> in Phase 3. Move to Phase 2.
<code>{"postgres":false}</code>	Service is up but can't reach the database. Either the migration didn't apply, or the <code>DATABASE_URL</code> env var is wrong.	Re-check Phase 1.4 (did the migration print <code>CREATE TABLE?</code>). In Render's dashboard, confirm <code>DATABASE_URL</code> appears in the service's Environment tab as <code><auto-wired></code> linked to <code>bedtime-story-db</code> .
KeyError: 'GEMINI_API_KEY' in Render's logs; service never reaches Live (so <code>curl /healthz</code> returns 502/503, not JSON) Connection times out for ~30s, then succeeds	The key wasn't set during Blueprint creation — the app reads it at startup, so the deploy fails before the port binds. Cold start (free-tier spin-up).	Phase 1.3 Moment B — go to the service's Environment tab and paste the key. Service auto-redeploys. Re-run <code>curl</code> once more — should be fast the second time. Don't treat the first 30-second wait as a bug.
Bad Gateway / 502 / 503	Service crashed at startup, log will show why (most often <code>KeyError</code> or <code>ImportError</code>).	Open the service's Logs tab, scroll to the most recent error, fix the root cause.

Cold-start caveat. Free-tier Render web services sleep after 15 minutes of no traffic. The first request after a sleep takes ~30 seconds while Render spins the container back up. That is expected free-tier behaviour, not a bug. Subsequent requests are fast (sub-second).

Phase 2 — Vercel (static frontend)

Vercel reads `vercel.json` at the repo root. There's no build step — `frontend/index.html`, `frontend/style.css`, `frontend/script.js` get served directly from Vercel's CDN.

Two paths to deploy on Vercel. This guide uses the dashboard import flow (browser-based, no CLI install). Vercel's docs as of 2026 also recommend a CLI path (`npm i -g vercel && vercel`) which auto-detects the project and deploys in one command. Both work; we use the dashboard for the curriculum because (a) it makes the connect-the-repo step visible (which is the conceptual lesson — Vercel reads from your GitHub, not from a local upload), and (b) it doesn't require students to install another CLI tool. If you're comfortable with terminals, `vercel --prod` from the repo root after `vercel login` is functionally equivalent.

2.1 Create the project

1. Sign in to <https://vercel.com>.
2. Go to <https://vercel.com/new> (or click Add New → Project from the dashboard — both lead to the same New Project page).
3. Import Git Repository → pick the same `bedtime-story-generator` repo from your GitHub. (If you don't see your repo, you may need to grant Vercel's GitHub app access to that specific repo via the Adjust GitHub App Permissions link.)
4. Vercel scans the repo and shows a configuration screen. Most fields are pre-filled from `vercel.json`:
 - Framework Preset: Vercel will auto-detect "Other" (no recognised framework — correct for plain HTML/CSS/JS). Leave it as-is.
 - Root Directory: leave as `./` (the repo root). `vercel.json`'s `outputDirectory`: `frontend` tells Vercel to publish from `frontend/` within that root.
 - Build and Output Settings: don't override. `vercel.json` already specifies the output directory. The build command stays empty (no build step for static files).
 - Environment Variables: leave empty. The frontend is static, so it doesn't need any env vars at deploy time. The `BACKEND_URL` is a JS constant in `frontend/script.js`, edited per-deploy in source. (This is a deliberate doctrine choice — see Module 7's Why this design in the crash course for the rationale.)
5. Click Deploy. Vercel deploys in ~30–60 seconds.

2.2 Get the Vercel URL

When the deploy finishes, Vercel shows a screen with confetti and your production URL — something like `https://bedtime-story-generator-XXXX.vercel.app`. Copy it.

If you open that URL right now, the page loads — but the Generate button errors out, because the JS is still pointing at `localhost:8000`. That's what Phase 3 fixes.

Phase 3 — Wire them together

The frontend needs to know the backend's URL. There's no env var injection at deploy time (we deliberately rejected build-step templating in the curriculum — see Module 7's Why this design in the crash course for the rationale). Instead, you edit one line in `frontend/script.js`, commit, push, and Vercel auto-redeploys.

3.1 Edit `BACKEND_URL`

In your local checkout of the repo:

1. Open `frontend/script.js`.
2. Find the line near the top that reads:

```
const BACKEND_URL = "http://localhost:8000";
```

3. Replace `http://localhost:8000` with your Render URL from Phase 1.5, no trailing slash:

```
const BACKEND_URL = "https://bedtime-story-api-XXXX.onrender.com";
```

3.2 Commit and push

```
git add frontend/script.js
git commit -m "Wire frontend to deployed Render backend"
git push
```

Vercel detects the push automatically (GitHub integration set up during Phase 2.1) and auto-redeploys in ~30 seconds. Open your Vercel project's dashboard and watch the new deploy land.

Phase 4 — Verify on a phone

The whole point of Module 7 is “an app on the internet that a parent on a different network can use.” The phone test is the moment that lands.

4.1 Open the Vercel URL on a phone

1. On your phone (NOT the demo laptop), open the Vercel URL in any browser.
2. The page should load — two-column layout on tablet, single-column on phone, soft cream pane for the story area.
3. Fill in the four form fields: child name, characters, setting, plot.
4. Tap Generate story.
5. Watch the story render in the right pane (or below the form on phone).

If it works: that’s the moment. “This is on the internet now. A parent on a different network, on a different device, just generated a bedtime story for a child.”

4.2 Confirm the story persisted

Back on your laptop, open Render’s database shell again (Phase 1.4) and run:

```
SELECT id, child_name, plot, LEFT(body, 60) FROM stories ORDER BY id DESC LIMIT 3;
```

The story you just generated on your phone should be the top row. The cross-origin path (browser on phone → Vercel CDN → JS → Render web service → Postgres) is now real.

4.3 Click-to-rehear (the carry-over from Module 6)

Generate a second story for the same child name. Then click the first story in the Past stories panel.

- Open DevTools (Chrome desktop: open the Vercel URL on your laptop browser, generate two stories, click an older one in the panel).
- DevTools → Network tab → click the older story.
- The story renders. No **/story** request fires. The body comes from the response of **/stories?child_name=...** (already cached in the panel item’s data-body attribute), and the click handler reads it from there.

That’s Module 6’s body-column-storage decision paying off in production — same words the child fell asleep to last night, zero Gemini call, zero non-determinism.

Common gotchas

Symptom	Cause	Fix
Vercel URL loads but Generate button errors with “Failed to fetch”	BACKEND_URL in script.js not updated, or has a typo (wrong protocol, trailing slash)	Re-check Phase 3.1
Browser DevTools shows CORS error blocking the /story request	Render service URL doesn’t match what the browser is calling, or CORS preflight rejected	Confirm BACKEND_URL is byte-identical to your Render URL; app/main.py allows ["*"] for V1
/story returns 502 with “Postgres is not reachable”	Migration not applied	Re-run Phase 1.4 in Render’s DB shell

Symptom	Cause	Fix
First request after 15 min idle takes 30+ seconds	Render free-tier cold start	Expected, not a bug — narrate it to the cohort, don't apologise
Render service appears, then errors with <code>KeyError: 'GEMINI_API_KEY'</code> in logs	Forgot Phase 1.3	Set the key in Render's Environment tab; service auto-redeploys
Vercel deploy fails with Node-build errors	Vercel detected <code>package.json</code> and tried to build it	Don't add <code>package.json</code> to <code>frontend/</code> ; <code>vercel.json</code> 's <code>buildCommand: null</code> is the directive
Vercel URL returns 500: <code>INTERNAL_SERVER_ERROR / FUNCTION_INVOCATION_FAILED</code>	Vercel auto-detected <code>requirements.txt</code> + <code>app/main.py</code> at the repo root and tried to deploy the Python backend as a Vercel serverless function (which then crashed because Vercel's runtime doesn't have your <code>GEMINI_API_KEY</code> or <code>DATABASE_URL</code>)	Two fixes — pick either: (a) dashboard: Vercel project → Settings → General → set Root Directory to <code>frontend</code> → Redeploy. (b) repo-side, ships with cohort: confirm <code>.vercelignore</code> exists at the repo root and excludes <code>app/</code> , <code>requirements.txt</code> , <code>Procfile</code> , <code>render.yaml</code> , etc. — push that file and redeploy. The repo's <code>vercel.json</code> also includes <code>framework: null</code> , <code>installCommand: null</code> to forcibly disable Python auto-detection.
404 for <code>/static/style.css</code> on the Vercel URL	Frontend is referencing the old Module 5 path	Module 7's <code>frontend/index.html</code> references <code>style.css</code> (not <code>/static/style.css</code>) — confirm you didn't carry the old path forward
Database becomes inaccessible after 30 days, deleted at day ~45	Render free Postgres 30-day expiry + 14-day grace period	Upgrade to paid Postgres before day 30 (or accept the loss + provision a fresh one for the next cohort)
Render free services suspended mid-month	750 instance-hour-per-workspace-per-month cap exceeded	Wait for next calendar month, or upgrade. Don't run multiple always-on free services in one workspace.
<code>psql: error: ... No such file or directory</code> when running the migration	The path you gave <code>psql -f</code> is relative to your current working directory, not to the repo root. You're in a folder where that path doesn't resolve.	Either <code>cd</code> to the right folder, or use a different path. From <code>dist/module_06_story_library/</code> or <code>dist/module_07_deploy_vercel/</code> , use <code>-f sql/002_create_stories.sql</code> . From the repo root, use <code>-f dist/module_07_deploy_vercel/sql/002_c</code> . The migration file lives in both places (byte-identical).

Symptom	Cause	Fix
Realised your External Database URL leaked (pasted in chat, committed by accident, etc.)	Render's External URL embeds the database password directly	Rotate immediately — Render dashboard → bedtime-story-db → Connect menu (or : menu) → Rotate Credentials. Old URL becomes invalid; web service auto-redeploys with new internal credentials. Takes ~30 seconds. See the security callout in Phase 1.4 for the full procedure.

What's NOT in V1's deploy

These are deliberate omissions, called out so you don't add them under pressure during a live demo:

- No build step on Vercel. Three static files. No npm, no node_modules/, no bundler. The lesson is “production deploy doesn't have to mean re-learning your stack.”
- No CDN-fronted backend. Render serves uvicorn directly. CDNs in front of dynamic APIs are a follow-on lesson.
- No tightened CORS. allow_origins=["*"] for V1. Production tightening to a specific Vercel domain is a named follow-up — not Module 7's lesson.
- No env-var injection at frontend build. The BACKEND_URL is a hardcoded JS constant edited per-deploy. PRD §1.5 rule 1: simplicity over demonstration. Build-step env injection would buy nothing the lesson requires and would re-introduce the build-tool concept Module 7 deliberately omits.
- No tests, no CI/CD beyond Vercel/Render's auto-deploy on push. Tests are a separate fundamental, deferred to a later curriculum. The Vercel-on-push and Render-on-push behaviour you get for free is enough CI for V1.

After you finish

If everything in Phase 4 worked, you have a real production app: backend on Render, frontend on Vercel, managed Postgres for the story library, child-safety constraints in the system prompt, deploy URLs you can share with anyone.

The Defend-It question (also in docs/crash_course.md Module 7) is worth sitting with before you call it done:

Why do we deploy the frontend as static files on Vercel and the backend as a long-running uvicorn process on Render — instead of running both on Vercel as serverless functions?

Three threads: V1 mental model preservation (uvicorn = receptionist who never goes home), connection-pool semantics on serverless (every cold start opens a new Postgres connection), and the fact that “deploy Module 7” should not be where you re-learn your stack. Reason through it; the answer is the curriculum's spine.

Doc verification provenance

This guide's facts about Render and Vercel were last verified against the live docs in May 2026. Specifically:

- Render Blueprint Spec (render.com/docs/blueprint-spec) — services:, databases:, fromDatabase, sync: false, runtime: python, plan: free.
- Render free tier (render.com/docs/free) — 30-day Postgres expiry + 14-day grace period; 750 instance-hours per workspace per calendar month; 15-min spin-down; ~30-second cold start.

- Render Postgres connection (render.com/docs/postgresql-creating-connecting) — Connect menu in top-right; External Database URL + Internal Database URL pattern; external `psql` from local terminal as the documented migration path.
- Vercel project configuration (vercel.com/docs/project-configuration) — `vercel.json` schema; `outputDirectory`, `buildCommand`, `framework` field names; vercel.com/new for dashboard import.

If you're maintaining this guide and it's been more than 6 months since the date above, re-verify before pointing a cohort at it — both vendors evolve their UIs and free-tier limits. The structural shape of the deploy (Render Blueprint + Vercel static + cross-origin wiring) is doctrine and unlikely to change; the specific numbers (free Postgres lifetime, instance-hour cap) and UI labels are the parts that drift.